Proyecto 3 - ITGSA

202400554 – Raúl José Daniel Paredes González

Resumen

El proyecto consiste en el desarrollo de un backend integral para la empresa Industria Típica Guatemalteca S.A. (ITGSA), implementando una API REST bajo el protocolo HTTP utilizando C# y .NET 10.0. La solución aborda la gestión automatizada de facturación electrónica y pagos bancarios para el canal de distribución "Tienda Virtual". Se aplicó el paradigma de Programación Orientada a Objetos para garantizar mantenibilidad y escalabilidad. El sistema procesa archivos XML incrementales: config.xml para clientes y bancos (con capacidad de insertar o actualizar), y transac.xml para facturas y pagos (solo inserción). La regla de negocio principal establece que los pagos se aplican a la factura más antigua del cliente, generando saldo a favor cuando corresponde. Se implementó un frontend en Razor Pages que consume la API, permitiendo carga de archivos, consulta de estado de cuenta con gráficas, y generación de reportes PDF. Se utilizaron expresiones regulares para limpieza de datos. El almacenamiento persistente se realiza mediante archivos XML.

Palabras clave

1. API REST
2. XMLG
3. raphviz
4. facturación
5. POO

Abstract

The project involves developing a comprehensive backend for Industria Típica Guatemalteca S.A. (ITGSA), implementing a REST API over the HTTP protocol using C# and .NET 10.0. The solution addresses the automated management of electronic invoicing and bank payments for the "Online Store" distribution channel. The Object-Oriented Programming paradigm was applied to ensure maintainability and scalability. The system processes incremental XML files: config.xml for clients and banks (with insert and update capabilities), and transac.xml for invoices and payments (insert-only). The main business rule stipulates that payments are applied to the client's oldest invoice, generating a credit balance when applicable. A Razor Pages frontend was implemented to consume the API, allowing file uploads, account statement queries with charts, and PDF report generation. Regular expressions will be used for data cleansing. Persistent storage is achieved through XML files.

Keywords

1. API REST

2. XML

3. Graphviz

4. billing

5. Object-Oriented Programming

Introducción

Brindar un panorama general del tema desarrollado revela la creciente necesidad de automatización en los procesos de facturación y pago dentro del sector comercial guatemalteco. La empresa ITGSA, al expandir su negocio hacia el canal digital "Tienda Virtual", requiere un sistema backend robusto que gestione transacciones financieras de manera eficiente. La importancia de este desarrollo radica en la integración entre plataformas de venta web y sistemas bancarios, permitiendo a los clientes realizar pagos desde sus cuentas bancarias con aplicación automática a facturas pendientes. Las bases teóricas adoptadas incluyen el paradigma de Programación Orientada a Objetos (POO), el protocolo HTTP para comunicación cliente-servidor, y el uso de XML como formato de intercambio de datos e insumos. Este ensayo tiene como propósito evidenciar la implementación técnica de una API REST, demostrando cómo el uso de expresiones regulares y almacenamiento persistente en XML resuelve los requerimientos planteados. ¿De qué manera una API bien diseñada puede optimizar los procesos de facturación y pago en una empresa en crecimiento? ¿Qué ventajas ofrece el uso de XML como base de datos?

1. Programación Orientada a Objetos en C#

La Programación Orientada a Objetos (POO) constituyó la base arquitectónica del sistema. Este paradigma permite organizar el código en torno a entidades del mundo real, facilitando su mantenimiento y escalabilidad. En palabras de Joyanes (2020), "la POO proporciona mecanismos de abstracción que permiten modelar problemas complejos mediante la identificación de clases y objetos".

2. Backend (API)

En el backend, se implementaron controladores que exponen endpoints REST para las operaciones de limpieza de datos, carga de configuración, carga de transacciones, consulta de estado de cuenta y resumen de pagos. El servicio DataService encapsula la lógica de negocio, incluyendo la regla crítica de aplicar pagos a la factura más antigua mediante ordenamiento por fecha y reducción progresiva del saldo pendiente, generando saldo a favor cuando el pago excede la deuda. Los helpers RegexHelper y XmlHelper gestionan la limpieza de datos (extrayendo NIT, fechas y valores numéricos de texto no estructurado) y la persistencia en archivos XML. Los archivos de entrada config.xml y transac.xml son incrementales: en configuración se permite actualización de clientes y bancos por NIT y código respectivamente; en transacciones solo se permite inserción, validando duplicados por número de factura y pagos por combinación de banco, cliente, fecha y valor.

3. Frontend (Razor Pages)

El frontend en Razor Pages consume la API mediante un cliente HTTP personalizado, permitiendo al usuario mediante una interfaz web amigable: resetear el sistema, cargar archivos XML con validación, consultar estado de cuenta por NIT o todos los clientes con visualización de transacciones ordenadas, y generar resúmenes de ingresos con gráficas de barras utilizando Chart.js. Los reportes pueden exportarse a PDF mediante la función de impresión del navegador. Se utilizaron expresiones regulares en el backend para extraer información relevante de campos que pudieran contener texto adicional, cumpliendo con la consideración del enunciado.

4. Procesamiento de Archivos XML

Los archivos XML constituyeron el medio de entrada y salida de información. El lenguaje de marcado extensible (XML) permite estructurar datos de manera jerárquica, facilitando su procesamiento automatizado (Pérez, 2020).

El archivo de entrada contiene la estructura siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<pacientes>
   <paciente>
      <datospersonales>
         <nombre>Lucas</nombre>
         <edad>33</edad>
      </datospersonales>
      <periodos>15</periodos>
      <m>10</m>
      <rejilla>
         <celda f="1" c="1" />
         <celda f="2" c="1" />
         <celda f="3" c="1" />
         <celda f="4" c="1" />
         <celda f="5" c="1" />
         <celda f="6" c="1" />
         <celda f="7" c="1" />
         <celda f="8" c="1" />
         <celda f="9" c="1" />
         <celda f="10" c="1" />
         <celda f="1" c="10" />
         <celda f="2" c="10" />
         <celda f="3" c="10" />
         <celda f="4" c="10" />
         <celda f="5" c="10" />
         <celda f="6" c="10" />
         <celda f="7" c="10" />
         <celda f="8" c="10" />
         <celda f="9" c="10" />
         <celda f="10" c="10" />
      </rejilla>
   </paciente>

</pacientes>
```

Figura 1. Entrada XML.

Fuente: Universidad San Carlos de Guatemala Facultad de Ingeniería Escuela de Ciencias y Sistema.

El sistema parsea este documento utilizando las clases XmlDocument y XmlNode de .NET, extrayendo la información para construir objetos Paciente y sus rejillas asociadas. Las celdas contagiadas se almacenan en una ListaCelda, asumiendo que las no mencionadas están sanas.

5. Visualización con Graphviz

Graphviz es una herramienta de código abierto para la representación gráfica de estructuras de datos mediante el lenguaje DOT (Takeyas, 2012). En este proyecto, se utilizó para generar imágenes de las rejillas en cada período de simulación.

El proceso consiste en:

1. Crear un archivo .dot con una tabla HTML que representa la rejilla.
2. Las celdas contagiadas se colorean en rojo.
3. Ejecutar el comando dot -Tpng para convertir el archivo a formato PNG.

Esta representación visual permite a los científicos observar directamente cómo evoluciona el patrón de contagio, facilitando la identificación de comportamientos periódicos. La Figura 1 muestra un ejemplo de rejilla generada.

Conclusiones

1. Se implementó exitosamente una API REST en C# con .NET 10.0 que cumple todos los endpoints requeridos, demostrando la viabilidad técnica de la solución.
2. La aplicación del paradigma POO permitió una arquitectura limpia y mantenible, separando responsabilidades en Controladores, Servicios, Helpers y Modelos.
3. Flexibilidad mediante XML: El uso de archvos XML para la configuración inicial y la generación de reportes brinda flexibilidad al sistema, permitiendo cargar diferentes configuraciones sin modificar el código fuente.
4. Visualización de datos: La integración con Grahviz proporciona una capa visual que facilita la comprensión de las relaciones entre drones y la secuencia de instrucciones optimizada.
5. El frontend en Razor Pages consume corectamente la API, ofreciendo una interfaz funcional para gestión de facturación, consultas y reportes gráficos en PDF.

Referencias bibliográficas

Joyanes,L. (2020). *Programación orientada a objetos con C#*. McGraw-Hill.

Pérez, C. (18 de diciembre de 2020). Qué es y cómo abrir un archivo XML. *MuyInteresante*. https://www.muyinteresante.es/tecnologia/23571.html

Takeyas,B. L. (2012). *Estructura de datos orientados a objetos: Pseudocódigo y aplicaciones en C#.NET*. Alfa Omega.

Microsoft. (2025). Documentación de .NET 10.0. Microsoft Learn.

Stubblebine, T. (2023). Regular Expression Pocket Reference. O'Reilly Media.

Anexos

1. Diagrama de Clases

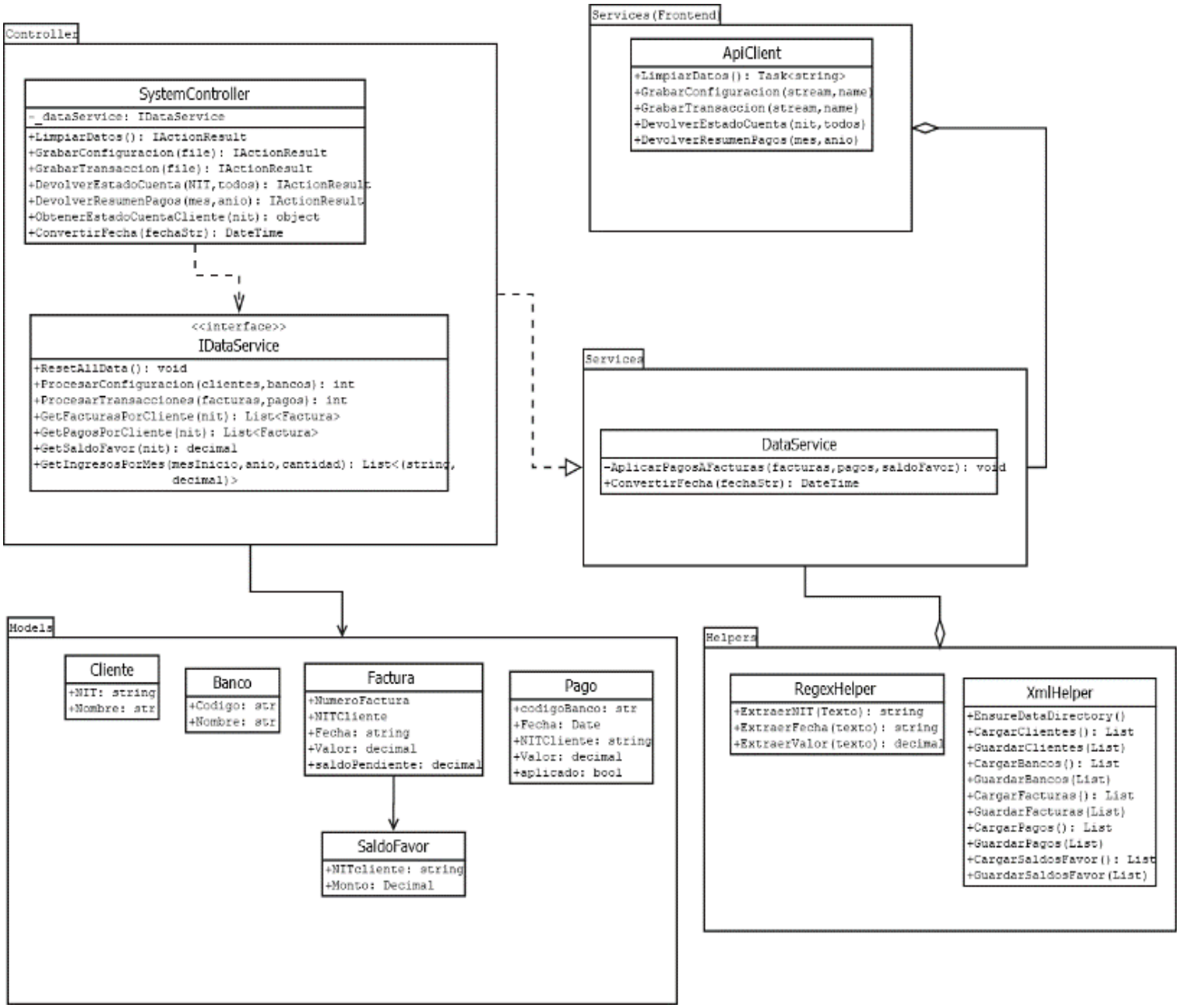

Figura 1. Diagrama de clases.

Fuente: Imagen propia.

2. Diagrama de Flujo

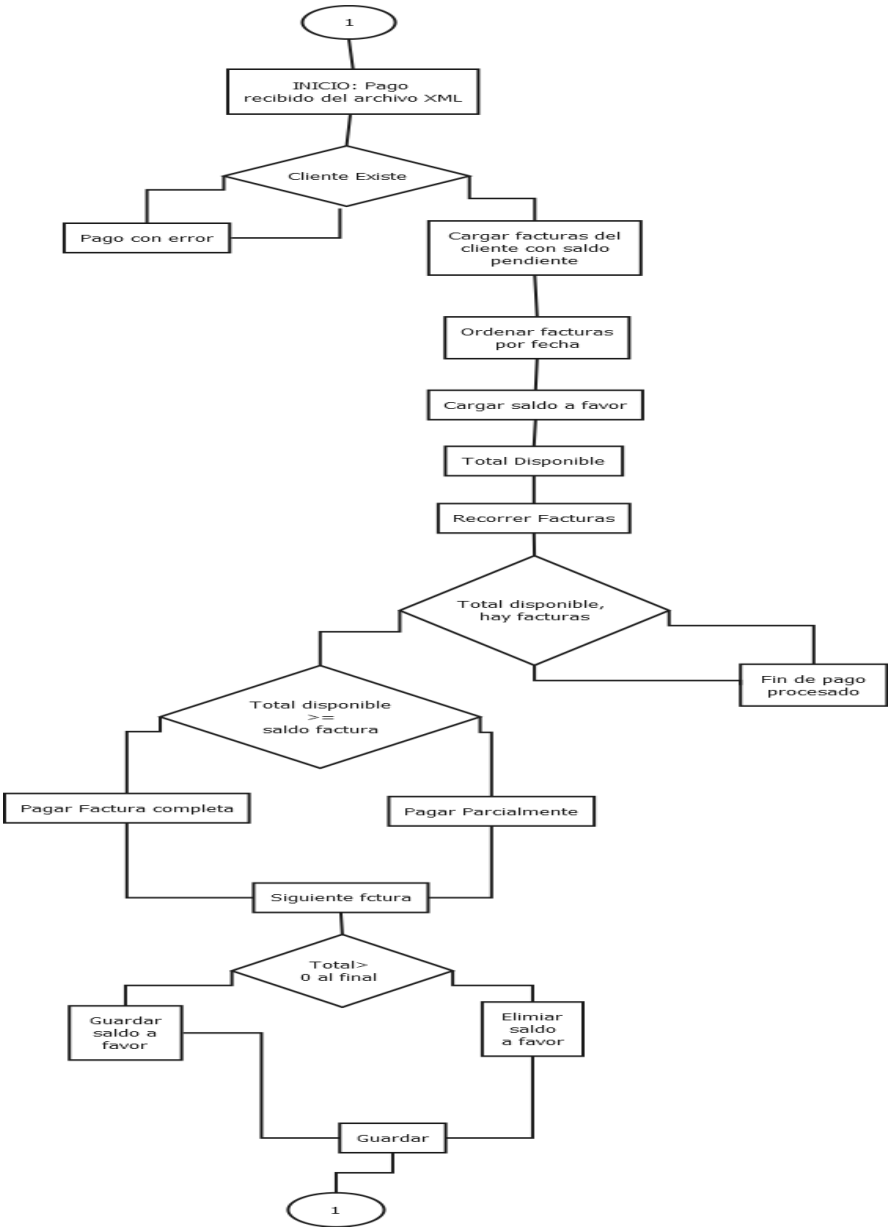

Figura 2. Diagrama de Flujo.

Fuente: Imagen propia.